Loan Offer Compass: An Auditable Comparison of Repayment Plans

Finance with AI Agents - Final Project Topic: Loans and financing (Appendix A) Kaiming Tian - HKUST IEDA - AI Agents and Financial Optimization 10 November 2026

Scope sentence. For a borrower comparing instalment-loan quotations, this project takes a CSV of offers plus a borrower affordability config, produces exact amortisation schedules, an IRR-based effective APR/EAR, an affordability verdict and a ranking, and is useful because headline nominal rates hide fees and term effects; I check it with a known-answer reference, an independent second implementation, cash-flow identities and changed-input cases.

HKUST IEDA · Finance with AI Agents · Final Project · 10 November 2026

1. The finance problem

A borrower shopping for a personal instalment loan receives quotations that are not directly comparable. One lender advertises a low nominal rate but charges an arrangement fee; another quotes a higher rate with no fees; a third offers a shorter term that lowers total interest but raises the monthly payment; a fourth uses an equal-principal plan whose payments decline over time. The borrower also has a hard monthly ceiling: an offer that is cheapest in total can be impossible to service.

User and task. An individual borrower (HKD 180,000, 36-month horizon in the primary case) who must choose among offers subject to a HKD 6,000 monthly budget cap and a 35% payment-to-income ceiling. The specific decision is: *which offer has the lowest all-in cost among the ones the borrower can actually afford, and how much principal could they borrow before the ceiling binds?*

Why this is useful. The usual approach - a hand-built spreadsheet or a "pick the lowest advertised rate" shortcut - is repetitive, error-prone and hard to verify. Common failure modes are: nominal rate confused with effective annual rate; fees omitted from the cost; a 24-month and a 36-month offer compared on total interest without regard to cash-flow timing; the first (largest) payment of an equal-principal plan used interchangeably with a level payment; and an offer ranked cheapest while being unaffordable. Each of these produces a number that looks plausible and is wrong.

Deliverable and smallest useful version. Inputs: one CSV of offers and one small JSON of borrower constraints. Outputs: a full schedule per offer, a comparison table, effective APR/EAR, an affordability verdict, a ranking and a machine-readable check record. The smallest useful version - three offers, one CSV in, one comparison table and one schedule out, with a known-answer check - was implemented first; the final version adds a second repayment structure, fees in two forms, a max-affordable-principal solver, charts, an independent checker and a reusable Skill package.

2. Data and method

2.1 Inputs and provenance

All inputs are **synthetic teaching data**, constructed by the author and versioned in the repository; no live quotes, paid data, downloads or personal information were used. The brief (Appendix B) explicitly permits course CSVs or synthetic examples. `data/source_notes.md` records provider, date, currency, units, period, retrieval (none) and transformations (none).

Input	Case A (primary)	Case B (fresh reuse run)
Offers file	`offers_case_A.csv`	`offers_case_B.csv`
Borrower config	`borrower_case_A.json`	`borrower_case_B.json`
Borrower	Individual, personal instalment loan	Small business, equipment finance
Currency (label only)	HKD	CNY
Principal	180,000	600,000
Terms	24 and 36 months	48 and 60 months
Nominal rates	5.20% - 6.80%	4.35% - 5.10%
Monthly budget cap	6,000	12,000
Monthly income / DTI ceiling	30,000 / 35%	60,000 / 30%
Binding monthly ceiling	6,000	12,000

The two cases deliberately differ in borrower type, currency, principal, term, rate level and fee mix so that the second run tests reuse rather than replay.

Check before analysing. `loan_compare.py` validates the header, required columns, non-empty file, unique non-empty `offer_id`, numeric and finite values, principal > 0, rate in [0, 1], whole-number term in [1, 600], non-negative fees, percentage fee <= 0.5, recognised `repayment_type`, `periods_per_year = 12`, and positive cap/income with a DTI ceiling in (0, 1]. Blank optional fee cells default to zero by a declared rule. Any other missing or out-of-range value **stops the run with an explicit list of row errors and exit code 3** - nothing is silently dropped, filled or invented. The actual check result for Case A is in `outputs/case_A/run_log.txt`: 4 offers, 0 validation errors, 0 duplicate ids.

2.2 Financial definitions

Monthly periods; `i = annual_nominal_rate / 12`. Fees are assumed withheld from the disbursement at t = 0 (paying them separately at t = 0 gives the same net cash flow). From the borrower's point of view:

```
t = 0        + (principal - upfront_fee)
t = 1..n     - (interest_t + principal_t + monthly_fee)


interest_t   = opening_balance_t * i
closing_t    = opening_balance_t - principal_t
outflow_t    = interest_t + principal_t + monthly_fee
PMT          = principal * i / (1 - (1 + i)^-n)          [annuity, i > 0]
PMT          = principal / n                              [i = 0]
```

Two repayment structures are supported: `annuity` (level total payment) and `equal_principal` (constant principal, declining total payment). The final instalment clears the remaining balance exactly, so no residual dust is left.

The **effective APR** is the IRR of those cash flows, found by bisection: `APR = 12 * i_IRR`; `EAR = (1 + i_IRR)^12 - 1`. This is the only figure that puts rates, fees and terms on one footing. **All-in cost** is the nominal sum `sum(outflows) - net_disbursement = total interest + upfront fee + n * monthly_fee`; it is deliberately *not* discounted. Both are reported because they answer different questions and can rank offers differently.

Affordability uses the **peak** monthly outflow against the binding ceiling `min(monthly_budget_cap, monthly_income * max_payment_to_income)`, plus the payment-to-income ratio. The peak (not the average) matters because an equal-principal plan is front-loaded. `max_affordable_principal` is found by bisection on the principal. The **naive baseline** - pick the lowest headline nominal rate - is recorded as the simple heuristic for comparison.

2.3 Implementation

`skills/loan-offer-comparison/scripts/loan_compare.py` (engine) and `verify.py` (independent checker) are pure Python 3.9+ **standard library** code, matching the T2/T3 "no extra packages" constraint. No network, key, token or model call is needed to rerun the calculation. Outputs are deterministic: CSV schedules (10 decimals), `summary.json`, `comparison.md`, two hand-built SVG charts and a `run_log.txt`. `scripts/run_all.py` reproduces both cases in one command and captures real stdout, stderr, exit codes and timestamps as evidence.

3. Actual agent use

The work was carried out by a **single agent** (the brief explicitly welcomes single-agent projects), following the six-part GOAL / INPUTS / METHOD / OUTPUT / BOUNDARIES / CHECKS prompt pattern from T2/T3. `docs/agent-workflow.md` records the prompt verbatim and the tool-use log. Concretely, the agent:

1. **Read inputs with tools** - extracted the 9-page project brief and the T1,

T2, T3 student guides from local PDFs, and inspected the machine (Python 3.13.14, Git 2.55, Word 16.0) and the existing course fork `Kaiming5762/finance-ai-mini-demo`.

1. **Planned before consequential execution** - fixed the cash-flow convention,

the four-offer synthetic case and the folder layout, and confirmed the two external actions (public GitHub push, Word-based PDF export) with the user.

1. **Wrote and executed code** - created `loan_compare.py`, `verify.py` and

`run_all.py`, then ran them in the terminal. `evidence/E1_case_A_run.txt` shows the real engine stdout; `evidence/E3_fresh_case_B_run.txt` shows the second case.

1. **Debugged with evidence** - the first independent verification failed with

`Newton IRR did not converge` (11/12 checks). Instead of hiding it, the agent printed the NPV/derivative iterates, found that the Newton solver broke on a tiny step before accepting a small NPV residual, and fixed it with a relative convergence criterion plus a step-size return. The corrected run reached **83/83 checks passed** (`evidence/E2_verification.txt`).

1. **Checked and packaged** - inspected the saved CSV/JSON/Markdown/SVG files,

wrote the independent verification record, generalised the workflow into a regular Skill (`SKILL.md`), and executed it on the fresh Case B.

1. **Produced submission artefacts and saved versions** - wrote this report and

exported PDF/DOCX with the local Word installation, created the source ZIP, then committed and pushed only `final-project/` files to the public fork.

Tool environment - honest labelling. The agent runtime available in this working session was **WorkBuddy AI**, a desktop agent using the same class of approved local tools (file read/write, terminal execution, Git) that the T1-T3 guides describe for WorkSwarm / JiuwenSwarm. The evidence records are the **actual tool outputs of that runtime**, not chat text and not screenshots of another product relabelled. The calculation itself is plain Python, so running `python scripts/run_all.py` inside the classroom WorkSwarm app produces the same files; if a WorkSwarm-specific

UI screenshot is required, capture the tool panel of that run. No claim of an unperformed WorkSwarm UI execution is made here.

4. Result

4.1 Primary case (Case A, HKD)

`evidence/E1_case_A_run.txt` is the actual engine output; `outputs/case_A/comparison.md` and `summary.json` are the saved results.

Offer	Plan	Nominal p.a.	Term	Peak/month	Upfront fee	Total interest	All-in cost	Effective APR	EAR	Affordable
A1_LOW_RATE_FEE	annuity	5.200%	36	5,410.94	4,500.00	14,793.82	19,293.82	6.900%	7.123%	YES
A2_NO_FEE	annuity	6.300%	36	5,500.45	0.00	18,016.18	18,016.18	6.300%	6.485%	YES
A3_SHORT_TERM	annuity	6.800%	24	8,042.75	0.00	13,026.09	13,026.09	6.800%	7.016%	**NO**
A4_EQUAL_PRINCIPAL	equal_principal	5.900%	36	5,915.00	1,200.00	16,372.50	18,652.50	6.741%	6.953%	YES

All currency figures are HKD. The engine's seven internal identity checks pass 7/7 for every offer.

Finding 1 - the headline-rate trap. The naive baseline picks `A1_LOW_RATE_FEE` (5.200%, the lowest quoted rate). Its 2.5% arrangement fee (HKD 4,500 withheld from the disbursement) raises the effective APR to **6.900%**, above the fee-free `A2_NO_FEE` at **6.300%**. Ranking by effective APR gives A2, A4, A3, A1; the shortcut and the all-in ranking **disagree**. A borrower following the shortcut would pay HKD 1,277.64 more in all-in cost over three years (19,293.82 vs 18,016.18).

Finding 2 - cheapest ≠ affordable. `A3_SHORT_TERM` has the lowest all-in cost of all four offers (HKD 13,026.09) because 24 months accrue less interest, but its peak instalment of HKD 8,042.75 exceeds the HKD 6,000 cap by HKD 2,042.75. It is correctly excluded by the affordability gate. Among **affordable** offers the ranking by all-in cost is A2 (18,016.18), A4 (18,652.50), A1 (19,293.82).

Finding 3 - front-loaded plans. `A4_EQUAL_PRINCIPAL` has the smallest budget headroom (HKD 85.00) because its first payment (HKD 5,915.00) is much larger than its last (HKD 5,054.58). Using an average payment would have understated the affordability risk. A controlled changed-input test (C6, same principal, rate and term, only the repayment structure changed) confirms the structural effect: equal-principal interest HKD 11,081.50 vs annuity HKD 11,403.20. Comparing A4's interest directly with A2's would be invalid because their rates and fee structures differ.

Affordability limits. A bisection solver gives the largest principal whose peak instalment fits the HKD 6,000 ceiling: A1 HKD 199,595.65, A2 HKD 196,347.59, A3 HKD 134,282.36, A4 HKD 182,599.83. This turns a yes/no verdict into an actionable borrowing limit.

4.2 Fresh reuse case (Case B, CNY)

`evidence/E3_fresh_case_B_run.txt` is the actual output of the reused Skill in a new, previously empty output folder.

Offer	Nominal p.a.	Term	Peak/month	All-in cost	Effective APR	Affordable
B1_INDUSTRIAL	4.350%	60	11,144.93	77,696.03	4.971%	YES

Offer	Nominal p.a.	Term	Peak/month	All-in cost	Effective APR	Affordable
B2_CITY_BANK	4.850%	60	11,281.55	76,893.21	4.850%	YES
B3_LESSOR	5.100%	48	15,050.00	68,475.00	5.629%	NO
B4_DIGITAL	4.600%	48	13,769.12	63,917.88	5.076%	NO

The pattern replicates on independent inputs: the naive pick B1 (4.350%) is worse on effective APR than the fee-free B2 (4.850%), and the cheapest offers in nominal cost (B3, B4) are unaffordable. The all-in ranking among affordable offers is B2 then B1. All 83 verification checks pass on this run too.

5. Evaluation and independent verification

The brief asks for a simple comparison or reference calculation, an independent check of an important result, and an honest account of failures and limitations. `outputs/case_A/verification.md` and `outputs/fresh_case_B/verification.md` hold the full 83-row pass/fail tables; `evidence/evaluation_record.md` is the compact Appendix F version.

5.1 Two genuinely different checking methods

The brief warns that "a self-check may repeat the original error; a second agent is not a substitute for a different checking method". `verify.py` therefore re-reads the **saved schedule CSVs** and the **original offers CSV** (not the engine's `summary.json` values) and uses different algorithms:

Quantity	Engine	Independent checker
Level payment	closed-form annuity formula	bisection on a recursive balance equation `b_t = b_(t-1)(1+i) - p` until `b_n = 0`
Monthly IRR	bisection on NPV	Newton's method with the analytic derivative `dNPV/dr = sum t * o_t / (1+r)^(t+1)`
Identities	computed in memory	recomputed from the saved schedule CSV rows

Cash-flow identities checked on saved numbers: `interest_t = opening_t * i`; `closing_t = opening_t - principal_t`; `outflow_t = interest + principal + fee`; sum of principal = original principal; final balance = 0; sum of outflows = interest + principal + all monthly fees; all-in cost = total interest + all fees = sum(outflows) - net disbursement; `NPV(i_IRR) = 0`; peak outflow and the affordability flag agree with the saved schedule. Maximum absolute residual observed was about `5e-11` currency units, against a `1e-6` tolerance.

5.2 Compact evaluation record

Case / input	Method / baseline	Expected / reference	Actual	Pass	Correction or limitation
Run integrity	saved summary vs offers CSV	4 offers	4	PASS	-

Case / input	Method / baseline	Expected / reference	Actual	Pass	Correction or limitation
Engine self-checks	7 identities per offer	28/28	28/28	PASS	Self-check, not independent
Known answer K1	10,000 at 12% p.a., 12 months vs spreadsheet `=PMT(1%,12,10000)`	PMT 888.487886783; total paid 10,661.854641401; interest 661.854641401	identical to 9 dp	PASS	Reference shares the textbook formula; bisection payment is the cross-check
Independent payment	bisection on balance recursion vs engine closed form	equal to 1e-6	equal (e.g. A2 5,500.449578 both)	PASS	-
Independent APR	Newton IRR vs engine bisection IRR	equal to 1e-8	equal (e.g. A1 0.069003688 both)	PASS	Newton initially failed; fixed (see 5.4)
NPV at independently solved IRR	NPV = 0 on saved outflows	<= 1e-6	<= 5e-10	PASS	-
Changed inputs C1-C6	rate +1pp; +2% upfront fee; 36->48 months; +50 monthly fee; cap 3,000 vs 5,000; annuity -> equal-principal	monotonic/finite changes as stated	e.g. fee adds exactly 2,400 to cost and lifts APR 5.990% -> 7.352% while payment is unchanged	PASS	Small sample of 6 cases, not proof of generality
Invalid inputs E1	8 malformed CSVs	clear `InputError`, no calculation	8/8 rejected	PASS	-
Error behaviour E2	CLI on bad CSV	exit 3, `INPUT ERROR`, no output folder	exactly as expected	PASS	-
Naive baseline	lowest headline rate vs lowest effective APR	disagreement exposes the trap	A1 vs A2 (Case A); B1 vs B2 (Case B)	PASS	Baseline is a heuristic, not a gold standard
Total	`verify.py` on each run	-	**83/83 passed, 0 failed** (both runs)	PASS	-

5.3 What the evaluation establishes - and what it does not

It establishes internal arithmetic consistency, agreement between two different implementations of the payment and the IRR, correct behaviour under changed and invalid inputs, and that the ranking claim follows from the saved schedules. It does **not** establish that the synthetic offers are real quotations, that any lender withholds fees at t = 0, that a borrower's rate stays fixed, or that the conclusion generalises beyond these eight constructed offers. Recalculation cannot authenticate source data. The naive baseline is a deliberately simple heuristic chosen before seeing results; no rules were tuned after observing the test outcomes. Because the sample is small and self-constructed, these results are evidence of a working, checkable procedure, not of a market-wide empirical finding.

5.4 A real failure and its correction

The first independent verification run reported **11/12 checks passed** with `AssertionError: Newton IRR did not converge` (`verify.py` exited 5). Printing the Newton iterates showed convergence in 3-4 steps - NPV fell from ~12,590 to ~1e-11 - but the stopping rule broke on `abs(step) <= 1e-15` *before* accepting a small NPV residual, then raised. The fix was a **relative** convergence criterion (`tol * net_disbursement`, appropriate because NPV is measured in currency units) and returning the iterate when the Newton step becomes tiny. After the fix all 83 checks passed, both for Case A and for the fresh Case B. The episode is retained here and in `docs/agent-workflow.md` because it is the clearest evidence that the checker can actually fail and detect a problem, rather than being a decorative always-green script.

6. Limitations

- **Monthly instalments only.** `periods_per_year` must be 12; quarterly or

daily products are refused with an explicit error. No intra-month compounding.

- **Fixed, simple products.** No variable rates, early repayment, refinancing,

grace periods, balloon payments, insurance, default/prepayment risk or taxes.

- **Nominal all-in cost is undiscounted.** It is not comparable across different

terms without the APR/EAR; the report shows both and notes where rankings differ (A3 is cheapest in nominal cost but unaffordable).

- **Fee timing is an assumption.** Fees are withheld from the disbursement. A

lender who finances the fee instead produces different cash flows.

- **Synthetic data.** No real quotes, no market data, no authenticity check.

`currency` is a display label; no FX conversion is performed, so Case A and Case B must never be compared against each other.

- **APR definition.** The IRR-based borrower cost is not asserted to be any

jurisdiction's statutory APR (e.g. a regulated disclosure figure).

- **Numerical.** Schedules are written at 10 decimals; residuals of ~1e-10

remain after round-tripping. Checks use a 1e-6 absolute tolerance.

- **Evaluation size.** 83 checks on two small self-constructed cases; a short,

clearly defined comparison, not a benchmark suite. No timing or token measurements were instrumented, so none are reported.

- **Single agent.** No team coordination was used; the reusable capability is a

regular Skill, not a Team Skill.

7. Reusable contribution

7.1 What is packaged

A **regular Skill** at `skills/loan-offer-comparison/` (the default route in Appendix D; a Team Skill is unnecessary because no agent coordination is part of the capability). It contains `SKILL.md` with YAML `name` and `description`, scope, inputs/outputs, financial definitions, the six-part procedure, checks, failure handling and limitations; `scripts/loan_compare.py` and `scripts/verify.py`; `examples/` with two lawful synthetic cases; and a `README.md` for a new user.

Generalisation steps actually taken: offer IDs, currency, case ID, cap, income and DTI ceiling are configurable through files and CLI flags; output folders are caller-supplied; no personal absolute path, cached answer, prior approval, secret, API key or model output is stored in the package; all checks and error handling were retained. Helper paths resolve relative to the loaded skill directory. It is pure standard library, so another WorkSwarm user needs only Python 3.9+. The licence is MIT and reused work is attributed.

7.2 Fresh-case execution

Following the README in a **clean setup** (new shell, previously empty `outputs/fresh_case_B/`, skill files listed and `SKILL.md` shown before execution), the Skill was run on Case B: a different borrower, currency, principal, term, rate level and fee mix. It produced the full output set and **83/83 verification checks passed** (`evidence/E3`, `evidence/E4`, `outputs/fresh_case_B/verification.md`). This is **honest self-reproduction by the same author**, not a classmate's independent reuse test; no peer reuse is claimed. A load-and-plan check alone would not have established execution, so the essential workflow was actually run and its outputs inspected.

7.3 Community record

The reusable source, README, lawful example, checks and this report are published in the **public GitHub repository** `https://github.com/Kaiming5762/finance-ai-mini-demo` under `final-project/`. The repository README contains the required openJiuwen link **https://github.com/openJiuwen-ai/jiuwenswarm**. The exact final commit URL, source ZIP, publication date and fresh-run evidence are recorded in `community/contribution-record.md`. Hub submission and upstream PR were **not attempted** (optional routes); no status is claimed for them.

8. Attribution and AI assistance

- Course materials used: *Finance with AI Agents Final Project* (30 October

2026, 9 pages) for the requirements and the Loans and financing topic; *HKUST T1 Student Guide* for the Git fork/commit/push workflow; *T2 Student Guide* for the six-part prompt, "check before analysing" rule and evidence-based troubleshooting; *T3 Student Guide* for the optional Skill packaging exercise and the load-and-plan distinction.

- No third-party code, dataset, tutorial output or open-source implementation

was reused. The annuity and IRR definitions are textbook finance. All scripts, examples, checks, charts and this report were written for this project.

- **Substantial AI assistance:** a WorkBuddy AI agent read the brief and guides,

designed the case, wrote and executed the scripts, diagnosed the Newton failure, packaged the Skill, drafted this report, exported it with the locally installed Microsoft Word 16.0, and created the Git commit and push. The student confirmed the two external actions, reviewed the outputs and remains responsible for all choices and claims.

- Every numerical claim in this report is traceable to a saved file

(`outputs/case_A/summary.json`, `comparison.md`, `verification.md`, `outputs/fresh_case_B/...`) and to an executable calculation or check. No number comes from unpinned chat text.

- Licence: MIT (inherits the course repository `LICENSE`).

9. Reproduction and final check

```
cd final-project
python --version                      # 3.9+
python scripts/run_all.py              # both cases + evidence
python skills/loan-offer-comparison/scripts/verify.py --battery-only --report battery.md
```

Open `evidence/E1`-`E4` for the real command/output/exit-code records, and `outputs/*/verification.md` for the full check tables. To change a small input, edit one CSV cell (for example A1 rate `0.0520` -> `0.0620`) and rerun into a new folder: the payment and APR must both rise. The 5-minute demonstration script is `demo/demo-script.md`.

Final checklist: the finance task and core output are clear and the files are accessible; evaluation and an independent check support the important claims and limitations are stated; a new-case execution demonstrates reuse; the GitHub package is public with the openJiuwen link; the source ZIP contains the needed scripts and no secrets, private data, environments or large caches; report, demonstration and source agree.

Appendix A. Submission artefacts

Requirement	File / location
Report PDF (>= 3 pages)	`report/report.pdf` (also `.docx`, `.html`, `.md`)
Repository URL + exact Commit URL + source ZIP	`community/contribution-record.md`, `final-project_source.zip`
Demonstration (5 min)	`demo/demo-script.md`, `demo/demo-slides.html`
Execution evidence (4 compact records)	`evidence/E1-E4`, `evidence/execution_evidence.md`
Community record + fresh-run evidence	`community/contribution-record.md`, `outputs/fresh_case_B/`
Reusable package	`skills/loan-offer-comparison/`